

ENGINEERING AUDIT & STRUCTURE ANALYSIS

Snap Repas Project Code Quality & Architecture Report

A comprehensive review of the Front-End architecture, structural hierarchy, and code safety metrics of the Czech local service marketplace.

Prepared For: Snap Project Team

Date: May 30, 2026

Analyst: Technical Lead & AI Partner

Disclaimer: This report is compiled based on automated structural checks, file-system parses, and manual code scans of custom files located under the /src directory. All metrics represent the state of the workspace as of May 30, 2026.

1. Executive Summary

The **Snap Repas** project represents a standard landing page and diagnostic tracking system built for a hardware repair services operation. Our structural analysis indicates that the code is written to high functional standards using **TypeScript** and **React** in a **Vite-powered Single Page Application (SPA)**.

The logic is split into standalone sections corresponding to elements of the landing experience. In addition, an interactive tracking system (`status-tracker.tsx`) simulates updates on client repair status. Crucially, the code demonstrates strong discipline: **no TypeScript any overrides are present**, indicating solid type-safety, and there are **zero debug lines (such as console.logs) or standard codebase TODO placeholders left in active custom files**.

● Core Technology Stack & Libraries

Framework / Library:	React 18+ (TSX Syntax)
Build Tooling:	Vite (Fast HMR & ESM Bundle execution)
Typing Language:	TypeScript (Strict Checking Enabled)
UI & Styling Boilerplate:	Tailwind CSS v3 + Shadcn CSS utilities
Asset Registry:	Lucide-React for rich visual icons

2. Directory & Structural Overview

The project codebase is divided into clear functional packages. Below is an abstract visualization of the directory architecture under ``/src``:

```
src/
├─ app.tsx           # Central layout & Section assembler
├─ main.tsx          # Client bundle mount point
├─ vite-env.d.ts     # Global configuration typings
├─ App.css / index.css # Global CSS stylesheets
├─ assets/           # Brand images, team avatars, award logos
├─ hooks/            # Global helper hooks (use-mobile, use-
toast)
├─ lib/              # Shared configurations (e.g. cn
tailwind-merge)
├─ components/
│   └─ figma/        # Specialized image-with-fallback custom
logic
│   └─ ui/           # Atomic design blocks (Shadcn
boilerplate imports)
│   └─ [Custom Sections] # Custom components (cenik, faq, hero,
sluzby...)
```

3. Structural Analysis & Quantitative Metrics

The codebase balances localized customization alongside atomic UI material. Our scan found **69 total resource files**. This includes **49 imported components within the Shadcn UI folder**, leaving **12 custom domain components**, **2 custom hooks**, and **1 sharing helper (utils.ts)** representing direct developer ownership.

Custom File Breakdown Table

Component Filename	Lines (Code/Total)	Size (KB)	Czech Copy Accent- Chars	Redundant 'use client'
components/hero.tsx	184 / 202	8.51 KB	36	YES
components/recenze.tsx	194 / 213	7.61 KB	98	YES
components/kontakt.tsx	170 / 183	9.20 KB	60	NO
components/status-tracker.tsx	150 / 166	7.62 KB	65	YES
components/faq.tsx	110 / 118	7.57 KB	407	YES
components/sluzby.tsx	95 / 103	4.09 KB	41	YES
components/cenik.tsx	94 / 102	4.20 KB	92	YES
components/Team.tsx	95 / 101	3.65 KB	54	YES

Type-Safety & Hygiene Highlights

- **0 any typings matched:** Full static typing integrity exists on customized files. No developer bypasses were observed.
- **0 console logs found:** Complete cleanliness of debug logs in custom files, satisfying production code safety frameworks.
- **0 standard TODO flags remaining:** No architectural tasks are marked outstanding inside custom component files.

4. Detailed Issues & Architecture Improvements

Issue A: Redundant Next.js directive `'use client'` in Vite SPA (Low Severity)

Nearly all custom components under `src/components/` specify `'use client'` at the top edge.

Context: `'use client'` is a Next.js App Router boundary indicator to separate server bundling from client bundling. Inside a purely client-side Single Page Application (such as this Vite project), everything is client-mounted. The statement is ignored, serving as visual noise.

Correction: Safely delete `'use client'` lines from custom modules during any subsequent batch edits.

Issue B: Hardcoded Multilingual Copy & Translatable Blocks (Medium Severity)

Czech strings (measured by high concentrations of accented character tokens, especially in `faq.tsx` and `recenze.tsx`) are written directly into the TSX UI layers.

Context: While perfectly fine for initial prototypes, embedding local copies within markup prevents localized SEO alternatives, stops translation scaling, and forces layout revisions for copy adjustments.

Correction: Isolate copy strings into a localized JSON config structure or adopt a localized loader framework (e.g., `i18next`) for robust dynamic copy control.

Issue C: Coupling of Simulation Logic & Static Mock Data in UI files (Medium Severity)

In `status-tracker.tsx`, step indicators, search lookup algorithms, and synthetic database records (`MOCK_REPAIRS`) are integrated inside the single

component file.

Context: When replacing mocks with live REST API backend endpoints, the developer will have to rip apart the layout code, risking regression errors. Mocks are currently bound inside components.

Correction: Abstract simulator actions into separate functions or custom fetch hooks under `src/hooks` or a `services/` directory.

Issue D: Case Naming Discrepancies in Custom Component Directory (Informational)

The `src/components/` directory blends kebab-case and CamelCase schemes: `proc-snap.tsx` and `status-tracker.tsx` versus capitalized `Team.tsx` .

Context: Inconsistent file capitalization scheme causes visual confusion and sometimes results in import breaking behaviors between case-insensitive file systems (Mac APFS/Windows NTFS) and production compile servers (typically case-sensitive Linux).

Correction: Standardize on PascalCase filenames across all React components, e.g. `StatusTracker.tsx` , `Cenik.tsx` , `ProcSnap.tsx` .

5. Prioritized Action Roadmap

To elevate the technical architecture of the **Snap Repas** project from prototype status to high-end enterprise robustness, we recommend following the ordered implementation sequence below:

1 Standardize Naming and Clean Redundant Directives (Short Term)

Rename active components to PascalCase convention matching standard React community defaults. Strip the Next.js visual boilerplate line `'use client'` as it is redundant under Vite.

2 Decouple Mock Services from UI Layout Layer (Medium Term)

Extract simulation arrays (such as mock repair codes) out of `status-tracker.tsx`. Store mockup information inside separate JSON entities or implement a lightweight API adapter client wrapper. This secures easier migration to real backends later.

3 Deploy Localized Static Configuration (Medium Term)

Create a `/locales/cs.json` and store the Czech landing strings inside it. Reference them dynamically or import the config block in React. This separates layout edits from basic copy adjustments.

4 Integrate Automated Linting & Style Formatting checks (Long Term)

Ensure `eslint.config.js` rules enforce naming standards and warn about Next.js import leftovers. Add a standard ESLint parse hook before committing files.

Snap Repas Audit Session complete

Generated dynamically using Desktop Commander Engine • May 2026